Documentation Interview Guide: BRD vs FRD vs PRD vs SRS — WariMitra

Document Identifier: WM-DOC-GUIDE-030

Classification: BA Core Competency Documentation & Interview Training Framework

Target Audience: Business Analysts, Product Managers, Technical Systems Analysts Facing Evaluators

1. Comparative Executive Overview

Document Artifact	Primary Purpose	Primary Target Audience	Phase in Lifecycle	Key Contents	WariMitra Project Example
BRD *(Business Requirements Document)*	Defines the **"WHY"** and business rationale, high-level objectives, scope, and expected ROI.	Project Sponsors, Executives, Steering Committee, Lead BA.	Initiation / Discovery Phase	Problem statement, business objectives, scope boundaries, stakeholder RACI, business KPIs.	Goal to reduce golden-hour medical response time during the 21-day Wari pilgrimage (`06_BRD/BRD.md`).
FRD *(Functional Requirements Document)*	Defines the **"WHAT"** the system must do from an operational behavior standpoint.	Developers, Architects, QA Testers, System Analysts.	Elaboration / Requirements Analysis	Detailed system flows, inputs, outputs, preconditions, exception paths, and Given-When-Then rules.	Specifications for 3-second hold SOS trigger, rate limiting, and Geohash proximity query (`07_FRD/FRD.md`).

Document Artifact	Primary Purpose	Primary Target Audience	Phase in Lifecycle	Key Contents	WariMitra Project Example
PRD *(Product Requirements Document)*	Defines the **"WHAT & FOR WHOM"** from a user, market, and product strategy perspective.	Agile Squad, Product Designers, Scrum Master, Stakeholders.	Product Discovery / Release Planning	Product vision, user personas, MoSCoW prioritization, user stories, release strategy.	Persona profiles (elderly farmer vs. police dispatcher) and MoSCoW ranking (`08_PRD/PRD.md`).
SRS *(Software Requirements Specification)*	Defines the **"HOW IT WILL BE BUILT"** technical and system constraints.	Software Engineers, DevOps, Database Administrators.	Architectural Design Phase	Hardware interfaces, API protocols, database schemas, performance latency SLAs.	Firebase Cloud Functions Node.js 20 specifications and Firestore indexes (`docs/SRS.md`).
User Stories	Captures a bite-sized piece of functionality delivering direct value to an end-user.	Agile Development Squad (Devs, QA, Product Owner).	Sprint Backlog / Iteration Planning	`As a... I want... So that...`, INVEST criteria, Gherkin acceptance tests.	`US-SOS-001` hold-to-activate emergency panic trigger (`09_USER_STORIES/USER_STORIES.md`).
Use Cases	Models user-system interactions	Systems Analysts, Solution	System Design & Modeling	Actor, trigger, preconditions, main flow,	`UC-SOS-01` Trigger Emergency Distress Alert

	interactions	Solution		main flow,	Distress Alert

Document Artifact	Primary Purpose	Primary Target Audience	Phase in Lifecycle	Key Contents	WariMitra Project Example
	through formal preconditions, main success, and alternate flows.	Architects, Enterprise Testers.		alternate flows, exceptions, postconditions.	(`10_USE_CASES/USE_CASES.md`).
Acceptance Criteria	Defines the conditions of satisfaction for a story to be accepted as "Done".	Developers, QA Automation Engineers, Product Owner.	User Story Refinement	Given-When-Then behavioral statements or checklist criteria.	Given user holds button for 3s, When threshold met, Then `createSOS` creates doc in `sosAlerts`.
RTM *(Traceability Matrix)*	Proves bidirectional completeness from business need down to executed test code.	QA Leads, Compliance Auditors, Program Managers.	Continuous Lifecycle	Matrix mapping Req ID $ \rightarrow$ User Story $ \rightarrow$ Source Code $ \rightarrow$ Test Case $ \rightarrow$ UAT.	Bidirectional mapping from `BR-001` down to `functions/src/index.ts:20` and `UAT-SOS-01`.

2. In-Depth Artifact Breakdown & Interview Defense

2.1 Business Requirements Document (BRD)

- **Common Interview Question:** "*What is the difference between a BRD and a Functional Specification?*"
- **Strong Interview Answer:**

> *"A BRD is vendor-neutral and technology-agnostic. It focuses entirely on the business problem, organizational goals, and financial or operational value—answering 'Why are we investing in this initiative?'. For example, in WariMitra, our BRD defined the objective of reducing emergency response times across one million pilgrims during the 250 km Pandharpur Wari without mandating specific database engines. In contrast, the Functional Specification details the precise behavioral and technical logic—such as requiring a 3-second hold physics gesture and a 5 per minute rate limit on Cloud Functions—to satisfy that business goal."*

2.2 Functional Requirements Document (FRD)

- **Common Interview Question:** "*What makes a great Functional Requirement?*"
- **Strong Interview Answer:**

> *"A great functional requirement must be unambiguous, testable, complete, and traceable. It should clearly define preconditions, user inputs, exact system operations, outputs, and exception paths. In WariMitra, rather than writing a vague requirement like 'The app must handle emergencies quickly', I formulated FR-SOS-001: specifying that the client must capture GPS coordinates, generate a UUIDv4 idempotency key, call the createSOS Cloud Function, and enforce a server-side cap of 5 alerts per minute per user, accompanied by formal Given-When-Then acceptance criteria."*

2.3 Product Requirements Document (PRD)

- **Common Interview Question:** "*When do you use a PRD instead of a BRD?*"
- **Strong Interview Answer:**

> *"A PRD is the primary artifact in product-led and modern agile environments. While a BRD is traditionally used in enterprise waterfall or client-vendor contracts to define business scope, a PRD bridges business vision with design and agile engineering. It centers on the end-user through detailed user personas, user journey maps, competitive analysis, MoSCoW prioritization, and phased release roadmaps. In WariMitra, our PRD framed the product around personas like Tukaram Patil (the 68-year-*

roadmaps. In WariMitra, our PRD framed the product around personas like Tukaram Patil (the 68-year-old rural farmer) and Inspector Vaishali Shinde (the police dispatcher), driving decisions like native Marathi localization and virtualized dispatch queues."

2.4 Requirements Traceability Matrix (RTM)

- **Common Interview Question:** *"Why is an RTM critical and how do you maintain it?"*
- **Strong Interview Answer:**

"An RTM ensures two things: first, that no business requirement is forgotten or omitted during development (forward traceability), and second, that developers do not build unapproved, out-of-scope code known as gold-plating (backward traceability). In WariMitra, I maintained an RTM linking every Business Requirement (BR-001) to its Functional Requirement (FR-001), User Story (US-SOS-001), exact source code file (functions/src/index.ts:20), and UAT test script (UAT-SOS-01). This provided 100% test coverage and assured stakeholders that all critical safety requirements were verified prior to go-live."